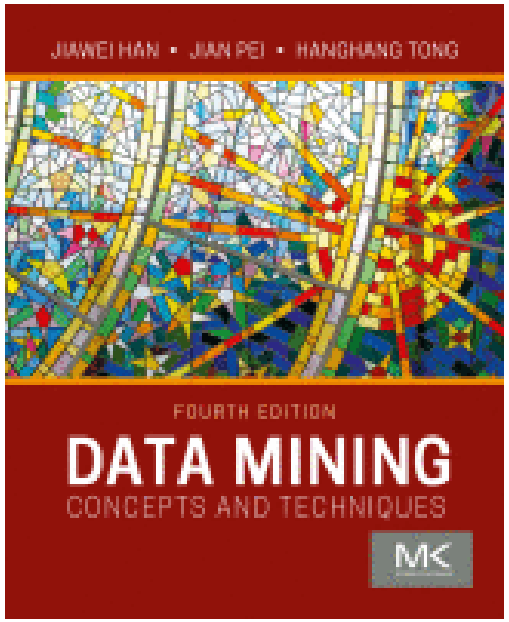
Deep Learning for Graph Data

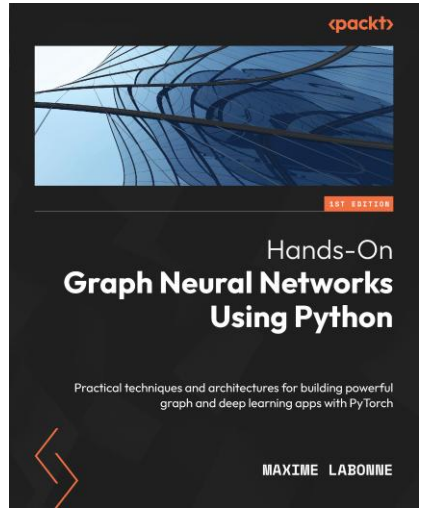
IF5141 dan IF4041 - Penambangan Data

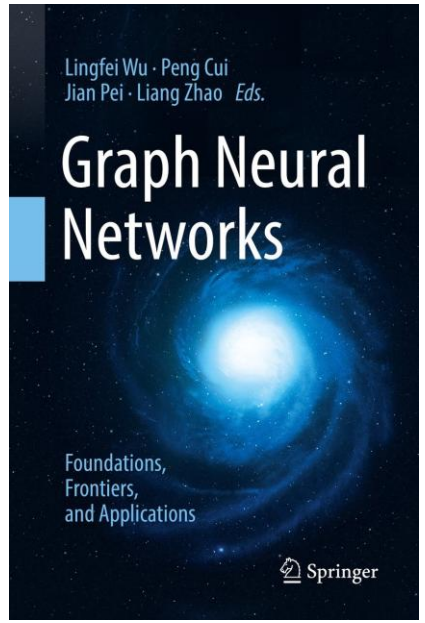
©2026 - Tim Pengajar IF4041 dan IF5141 Semester I TA 2026/2027

References

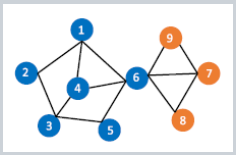
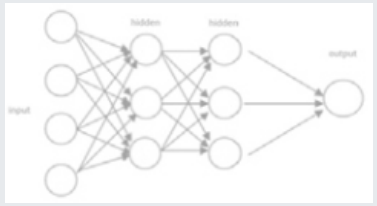
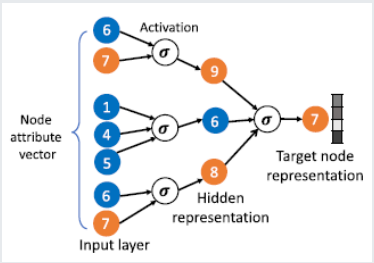
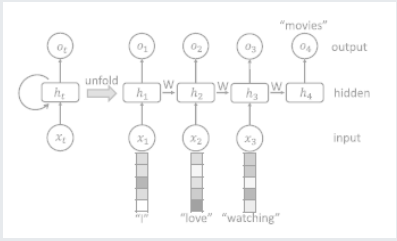
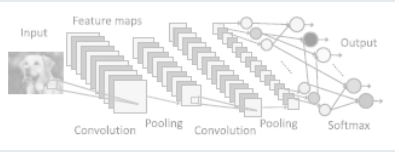
- Han, J., Pei, J., & Tong, H. (2024). *Data mining: Concepts and Techniques*. Morgan Kaufmann. https://doi.org/10.1016/C2013-0-18660-6
- Labonne, M. (2023). *Hands-On Graph Neural networks using Python: Practical techniques and architectures for building powerful graph and deep learning apps with PyTorch*. Packt Publishing Ltd.
- Wu, L., Cui, P., Pei, J., Zhao, L., & Song, L. (2022). *Graph Neural Networks: foundations, frontiers, and* applications. Springer Singapore. https://doi.org/10.1007/978-981-16-6054-2







Overview of deep learning architecture

Data Type	*Multidimensional data* (i.e., tabular)	*Relational* data (i.e., graph)	*Sequence* data (i.e., text)	*Grid-like* data (i.e., image)
			x = "I love watching movies."	
Deep Learning (DL) Architecture	Feed-forward neural networks 	Graph neural networks 	Recurrent neural networks 	Convolutional neural networks 

Another kind of deep neural network (which is designed for other types of input data) is a graph neural network (GNN) for *relational* (i.e., graph) data.

Data Type

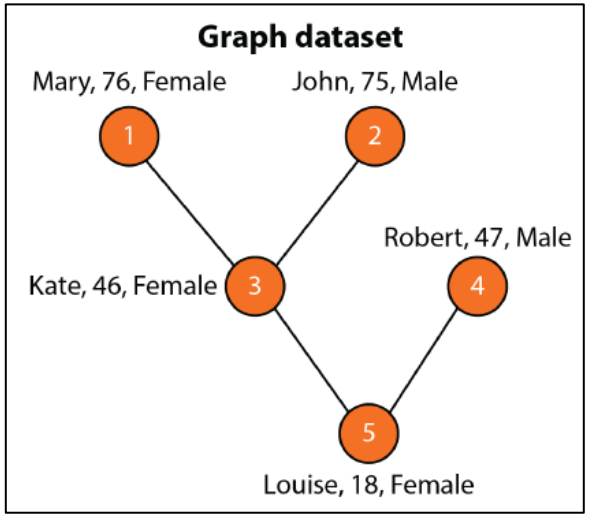
Graph vs tabular: Why Graph Data is Different

Tabular Data vs Graph Data

- Traditional tabular datasets, such as spreadsheets, represent data as rows and columns, with each row representing a single data point.
  - However, in many real-world scenarios (e.g., family tree), the relationships between data points are just as meaningful as the data points themselves.
- Graph datasets represent data points as nodes in a graph and the *relationships between those data points* as edges.
  - The connections between nodes are crucial in understanding the data (e.g., which of them is the parent of Louise?)

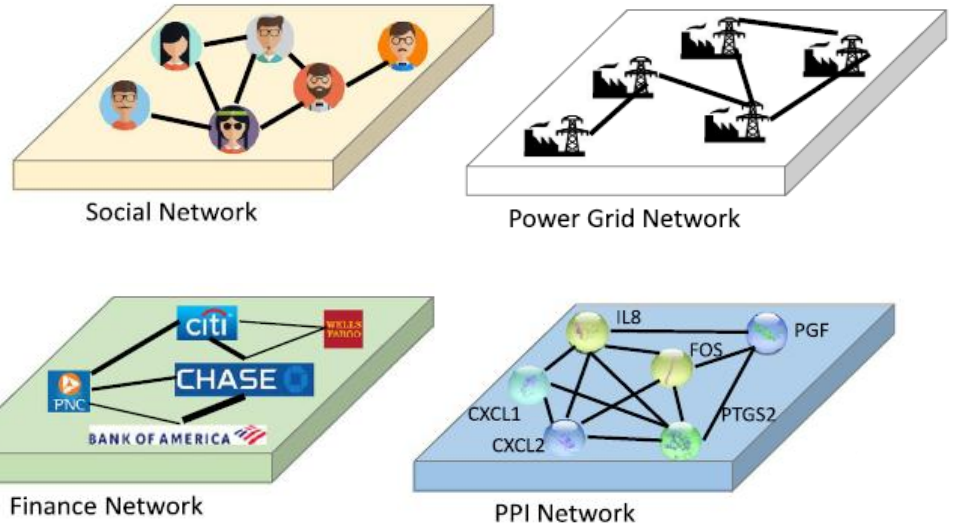
Tabular dataset

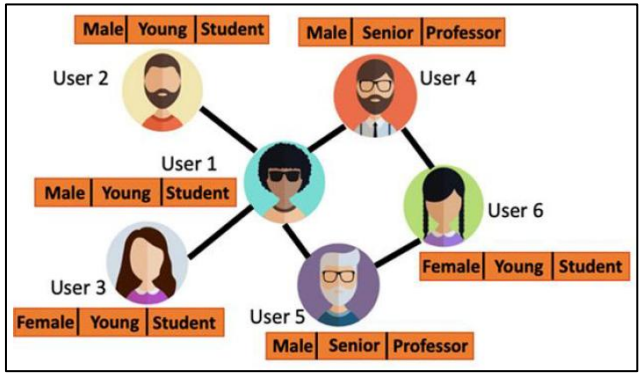
ID	Name	Age	Gender
1	Mary	76	Female
2	John	75	Male
3	Kate	46	Female
4	Robert	47	Male
5	Loise	18	Female



Graph Data

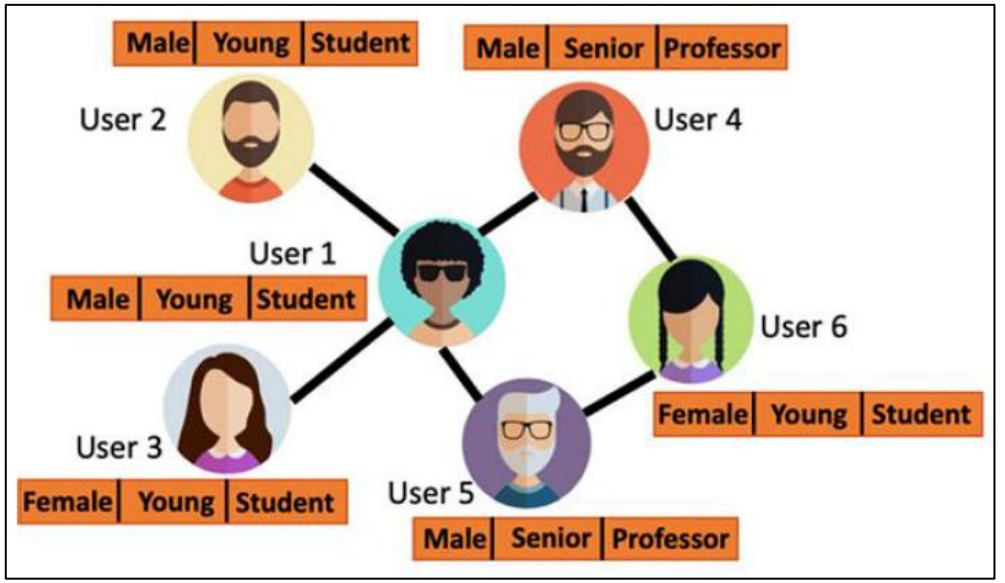
- **Graph data** (also referred to as **network data**) is essentially a collection of *nodes* (or vertices) linked with each other by *edges*. For example:
  - in a **social network**, nodes are users and edges represent the friendship between two users;
  - in a **power-grid**, nodes are power plants and edges are the power lines connecting two power plants;
  - in a **financial transaction network**, nodes are financial accounts and edges are the transactions between two accounts;
  - in bioinformatics, nodes of a **protein-protein interaction (PPI)** network are proteins and edges indicate the interactions between different proteins.
- Nodes or edges of a graph might have attributes.





Graph Data (cont.)

- A graph with n nodes can be represented by its adjacency matrix $\boldsymbol{A}$ of size $n \times n$. For example: Consider an undirected social network with 6 users (as 6 nodes) ➔ We represent it by a 6×6 adjacency matrix $\boldsymbol{A}$.
  - Each row and column of the adjacency matrix $\boldsymbol{A}$ represent a user.
    - E.g., the first row and column represent User 1; the second row and column represent User 2, etc.
  - Each entry of the adjacency matrix $\boldsymbol{A}$ indicates whether the two corresponding users are connected.
    - For example, since User 1 and User 3 are connected, we have that $\boldsymbol{A}(1, 3) = \boldsymbol{A}(3, 1) = 1$;
    - since User 2 and User 5 are not connected, we have that $\boldsymbol{A}(2, 5) = \boldsymbol{A}(5, 2) = 0$.



a graph (social network)

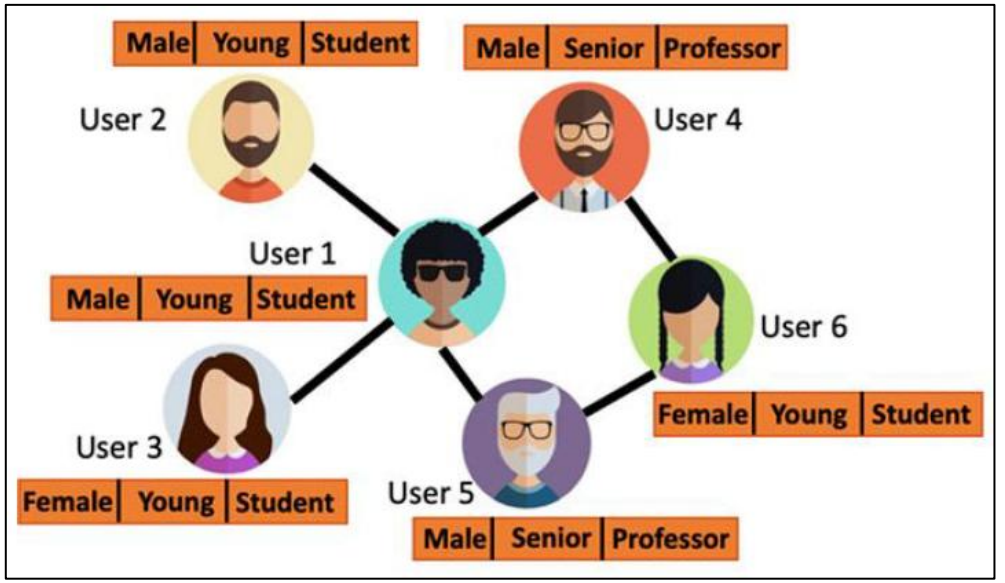


0	1	1	1	1	0
1	0	0	0	0	0
1	0	0	0	0	0
1	0	0	0	0	1
1	0	0	0	0	1
0	0	0	1	1	0

adjacency matrix $\boldsymbol{A}$

Graph Data (cont.)

- Nodes or edges of a graph might have attributes.
- Their attributes can be represented as a node (or edge) attribute matrix $\boldsymbol{X}$.
  - Rows are nodes (or edges). E.g., each row represents a user (or node).
  - Columns are attributes. For example:
    - The first column of $\boldsymbol{X}$ represents whether the user is male (1) or female (0).
    - The second column of $\boldsymbol{X}$ represents whether the user is young (1) or senior (0).
    - The third column of $\boldsymbol{X}$ represents whether the user is a student (1) or a professor (0).
  - Entries of the matrix are the attribute values. For example:
    - The first user is a young male student, we have that $\boldsymbol{X}(1, 1) = 1$ (male), $\boldsymbol{X}(1, 2) = 1$ (young), and $\boldsymbol{X}(1, 3) = 1$ (student).



a graph (social network)



Male	Young	Student
1	1	1
1	1	1
0	1	1
1	0	0
1	0	0
0	1	1

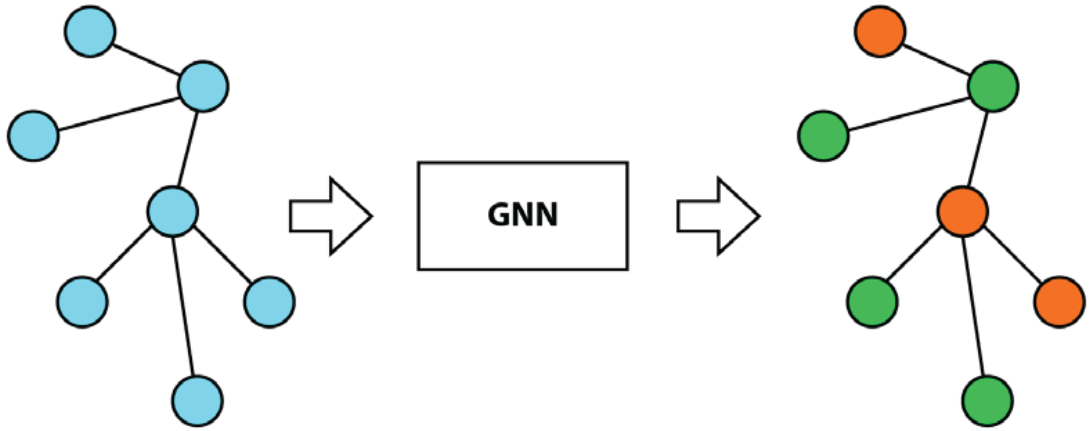
attribute matrix $\boldsymbol{X}$

Task using Graph Data

Node classification, Link prediction, Graph classification

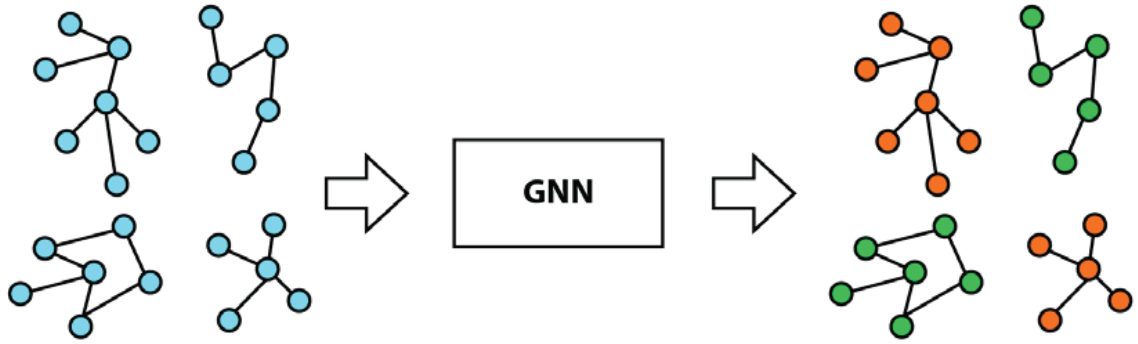
Node classification

- Predicting the category (class) of a node in a graph.
- E.g., categorize online users or items based on their characteristics.
- In this task, the model is trained on a set of labeled nodes and their attributes, and it uses this information to predict the class of unlabeled nodes.



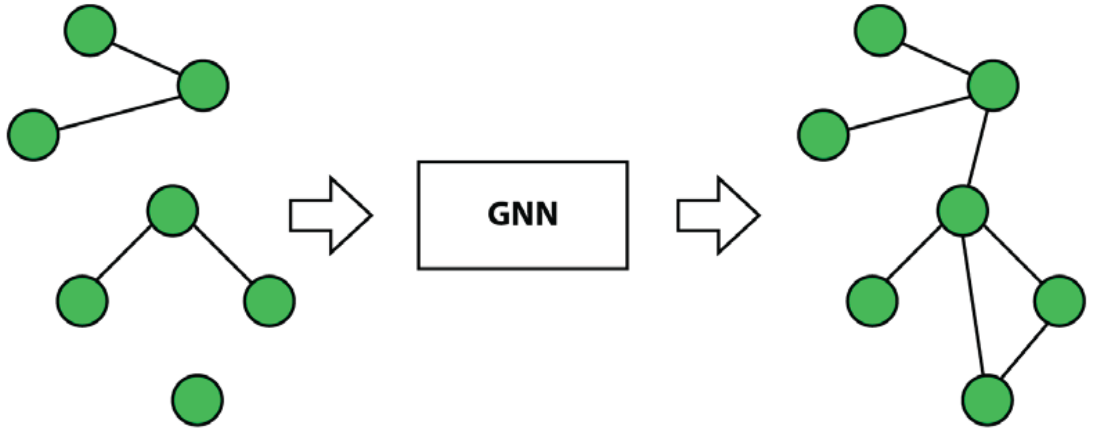
Graph classification

- Categorizing different graphs into predefined categories.
- In molecular biology, where molecular structures can be represented as graphs, the goal is to predict their properties for drug design.
- In this task, the model is trained on a set of labeled graphs and their attributes, and it uses this information to categorize unseen graphs.



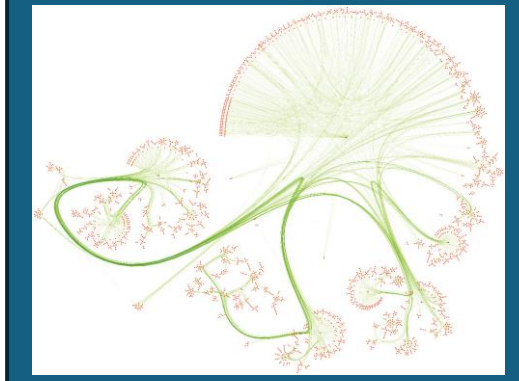
Link prediction

- Predicting missing links between pairs of nodes in a graph.
- E.g., predict the relationships between people based on their social network connections (friend recommendation).

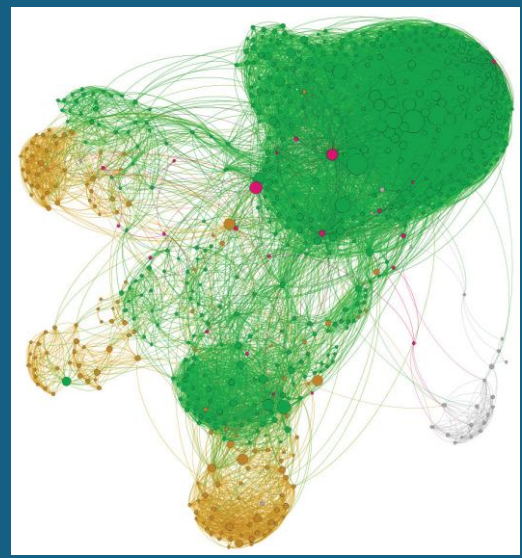


Graph Dataset Example

- **The Cora dataset:**
  - 2,708 nodes represent publications, where each connection is a reference.
  - Node features are described as a binary vector indicating the absence or presence of 1,433 unique words.
  - Our goal is to classify each node (publication) into one of seven categories.
- **The Facebook Page-Page dataset:**
  - 22,470 nodes represent an official Facebook page, where each connection is mutual likes between them.
  - Node features (128-dim vectors) are created from textual descriptions written by the owners of these pages.
  - Our goal is to classify each node into one of four categories: politicians, companies, television shows, and governmental organizations.



The Cora dataset

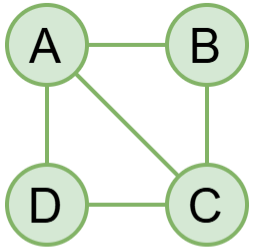


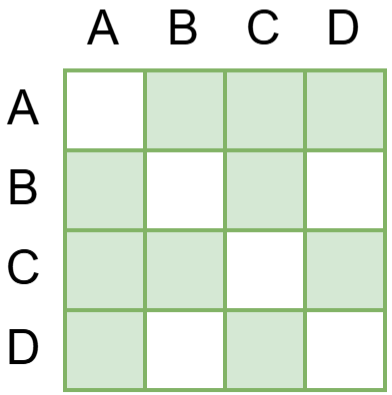
The Facebook Page-Page dataset

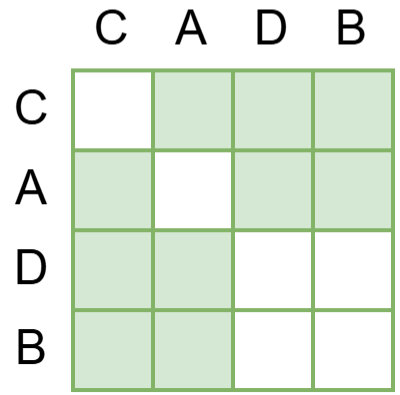
Why Standard MLP is Not Enough

MLP vs GNN

Permutation Invariance Challenge

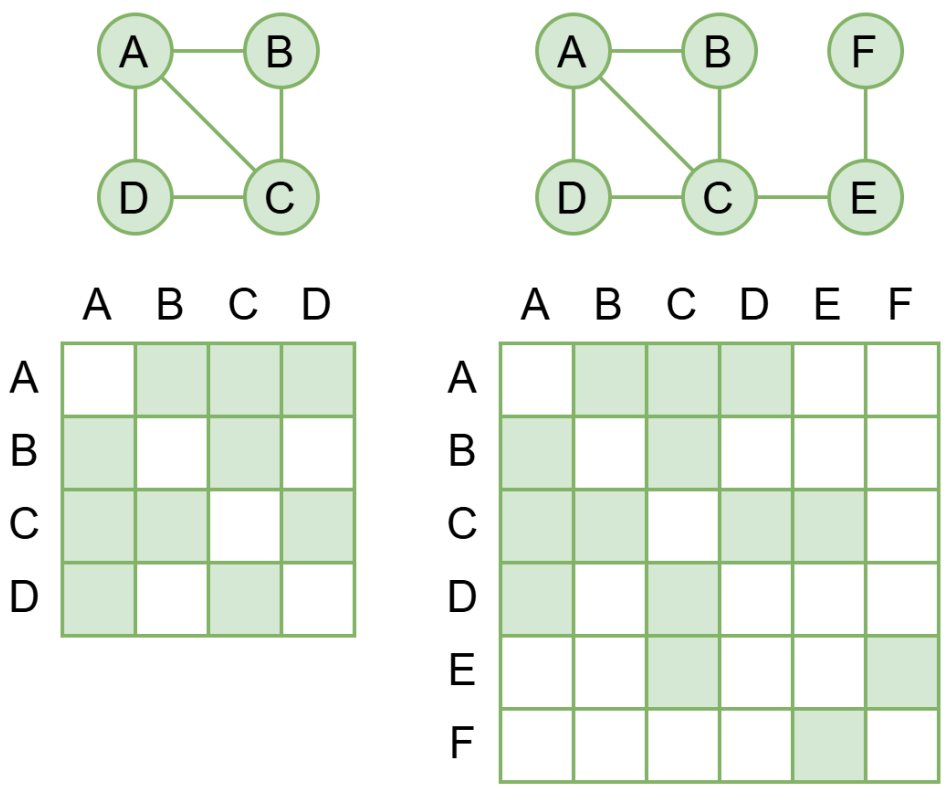




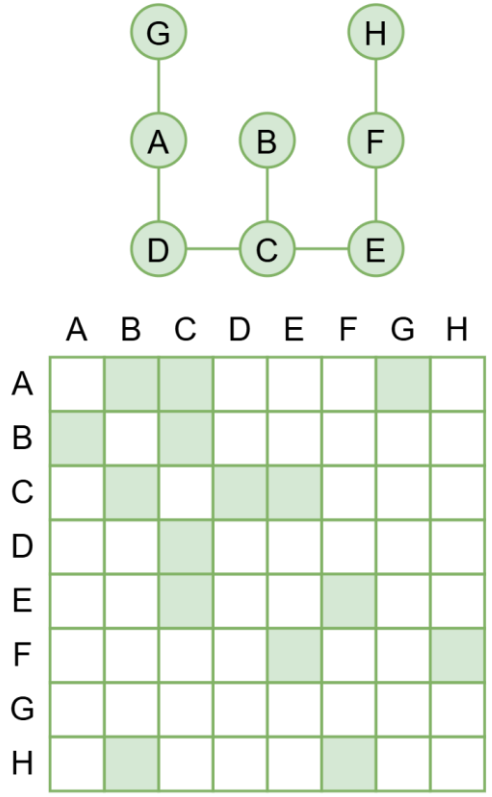


Same graph structure, different matrix representation!

Graph Size and Sparsity Challenge



Graph size explodes fast (N^2)!



Huge matrix, mostly zeros!

Node Classification Task

	MLP	GNN
Cora	53.47% (±1.81%)	74.98% (±1.50%)
Facebook	75.21% (±0.40%)	84.85% (±1.68%)

Using MLP (Multilayer Perceptron)

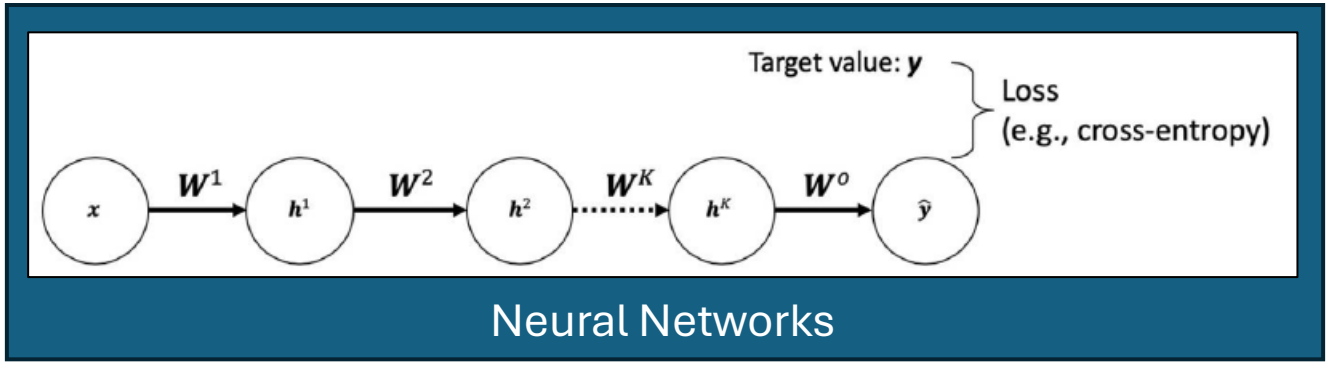
- Consider node features (adjacency matrix as a regular tabular dataset).
- Train, evaluate, and test the MLP model to classify our nodes.
  - The same experiment is repeated 100 times for each model on each dataset.
- Result: lower accuracy
  - Because it does not take into account the topology of the network.

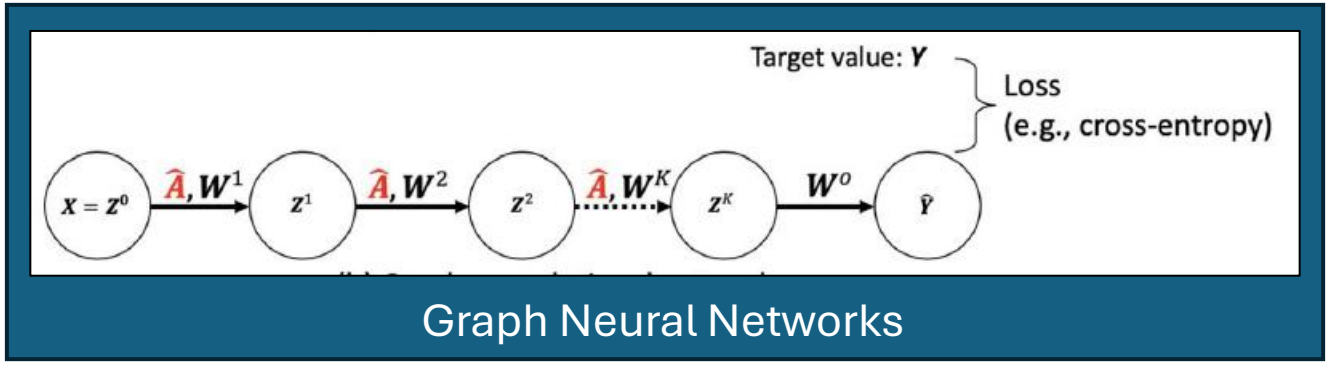
Using GNN

- Considers node features and also the entire neighborhood of each node (adjacency matrix with special handling)
- Train, evaluate, and test the GNN model to classify our nodes.
  - The same experiment is repeated 100 times for each model on each dataset.
- Result: higher accuracy
  - Because it takes into account the topology of the network ➔ from the adjacency matrix

NNs vs GNNs (Sequence Memory)

- The main difference lies in the aggregation step ($\widehat{A}$) in GNNs.





Learning Representations on Graphs

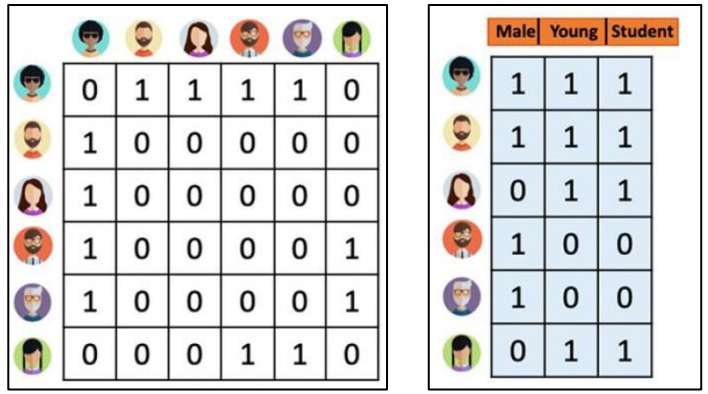
Node embedding

Why need node representations?

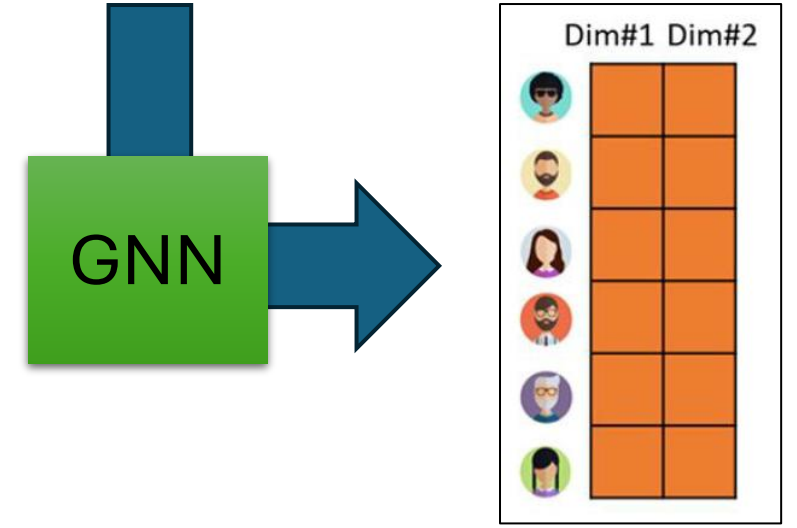
- The node classification goal is to classify the nodes into a few predefined categories. For example:
  - in social networks, we want to predict the political bias of each user;
  - in protein-protein interaction networks, we are interested in predicting the function role of each protein;
  - in the World Wide Web, we may have to classify web pages into different semantic categories.
- To make effective prediction, a critical problem is to have very effective node representations, which largely determine the performance of node classification.

Node Embedding

- Given an adjacency matrix of the input graph $\boldsymbol{A}$ with the node attribute matrix $\boldsymbol{X}$, a GNN produce the node embedding matrix $\boldsymbol{Z}$ whose rows are the embedding of the corresponding nodes.
- The node embedding can be used as the input to a variety of downstream applications, such as **classification, clustering, outlier detection, and link prediction**.



adjacency matrix $\boldsymbol{A}$, attribute matrix $\boldsymbol{X}$



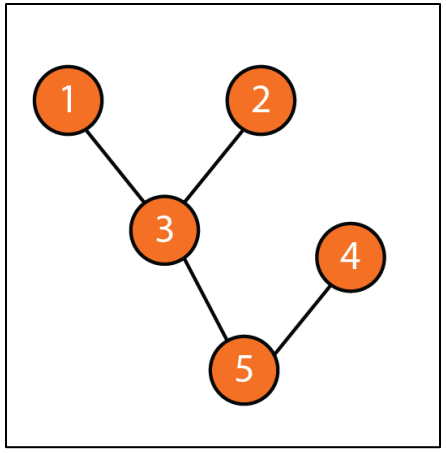
node embedding $\boldsymbol{Z}$

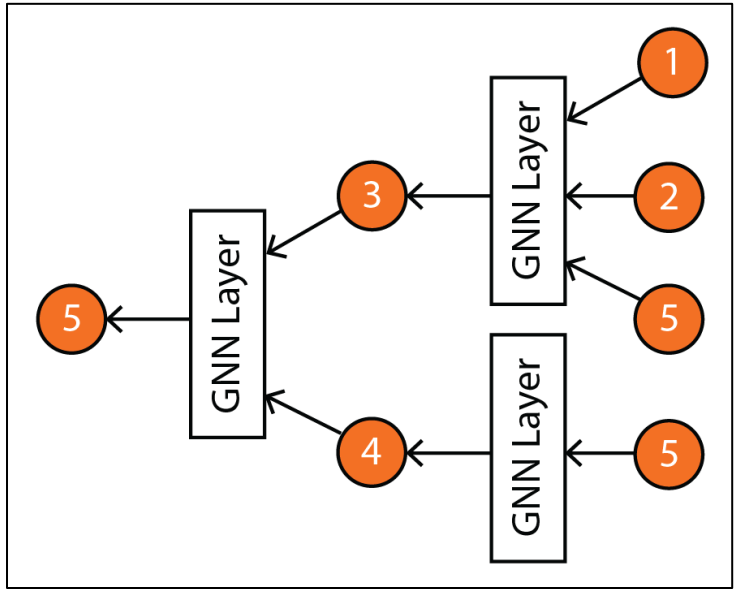
Main GNN Concepts: Message passing

How can a neural network learn from a node's neighbors?

Main GNN Concepts

- GNNs aggregate information from the neighboring nodes and edges to update the node representation. This operation is performed iteratively.
  - Left image: graph dataset.
  - Right image: simplified illustration of how a GNN computes the representation of “node 5” (in the graph data) based on its neighbors.





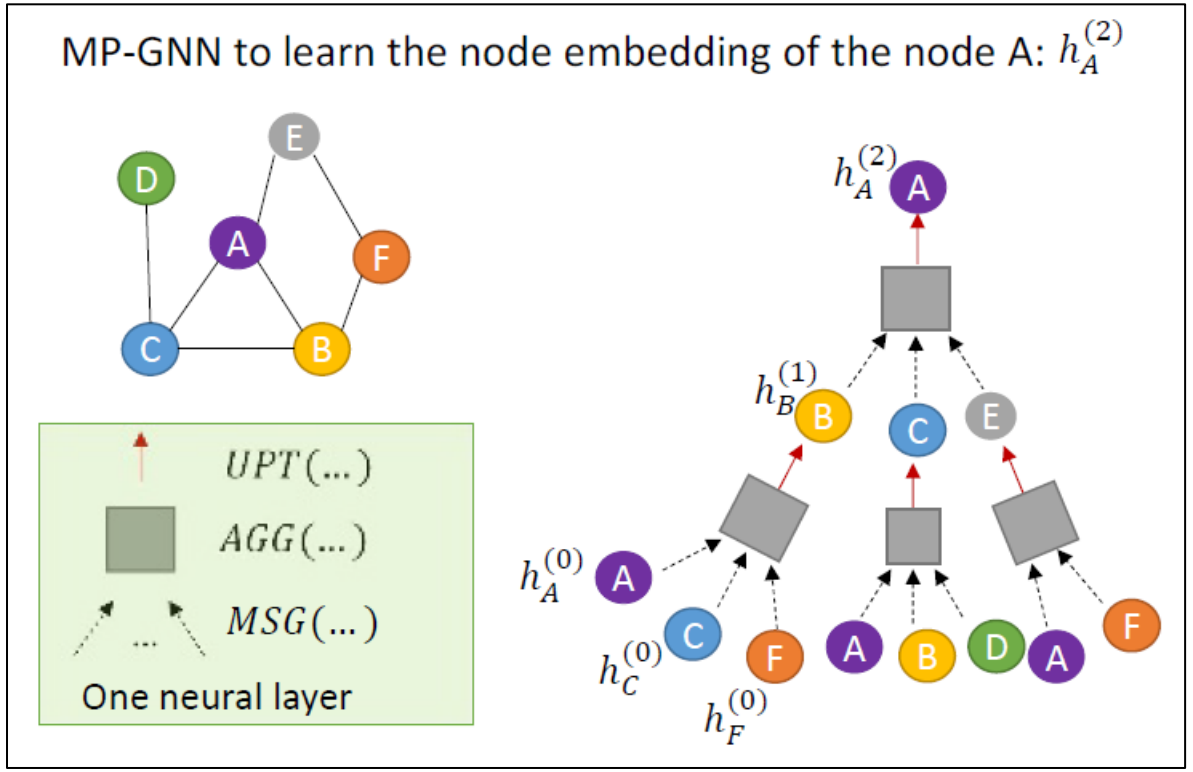
General Framework of GNNs

- In each layer we have two important functions:
  - **AGGREGATE** function:
    - A summation of all the messages from the neighbors.
    - To aggregate the information from the neighbors of each node.
  - **COMBINE** function:
    - The “node updating” function.
    - To update the node representations by combining the aggregated information from neighbors with the current node representations.

Wu, et al (2022)

Message Passing (MP)

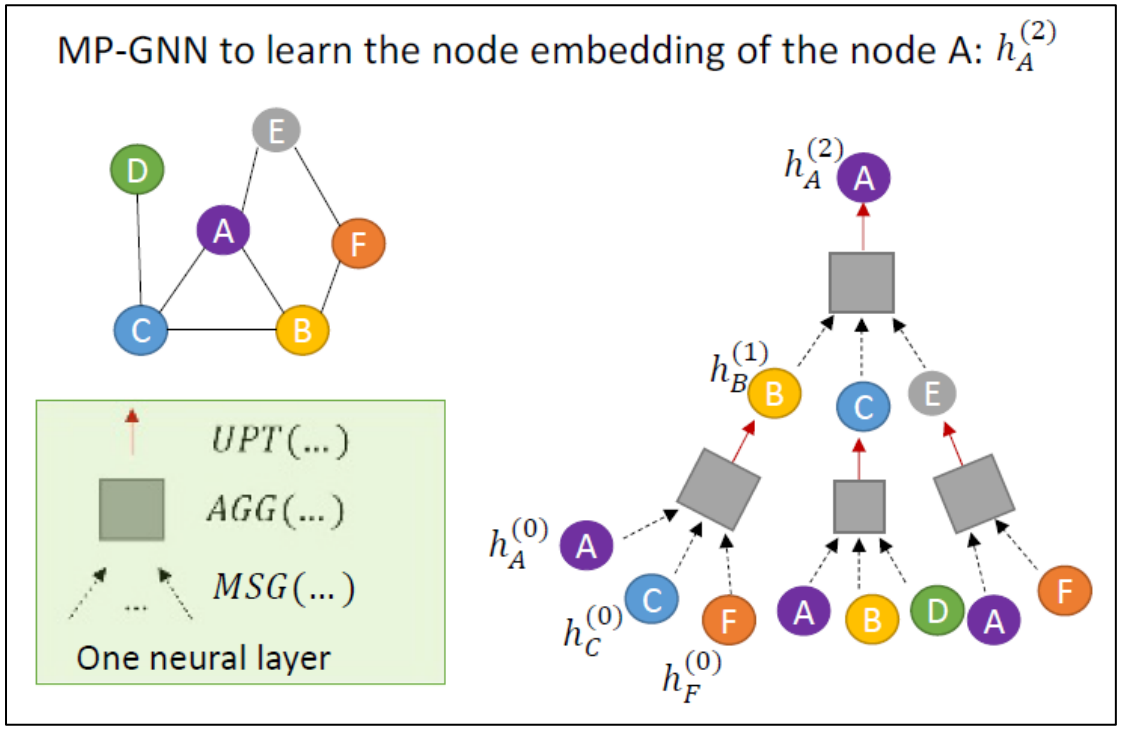
- Message passing is the most widely used framework to build GNNs.
- In each layer, perform:
  - the message (MSG) function,
  - the aggregate (AGG) function, and
  - the update (UPT) function.
- Most GNN architectures (e.g.: GCN, Graph-SAGE, GAT, GIN, etc.) adopting specific MSG, AGG, and UPT operations.



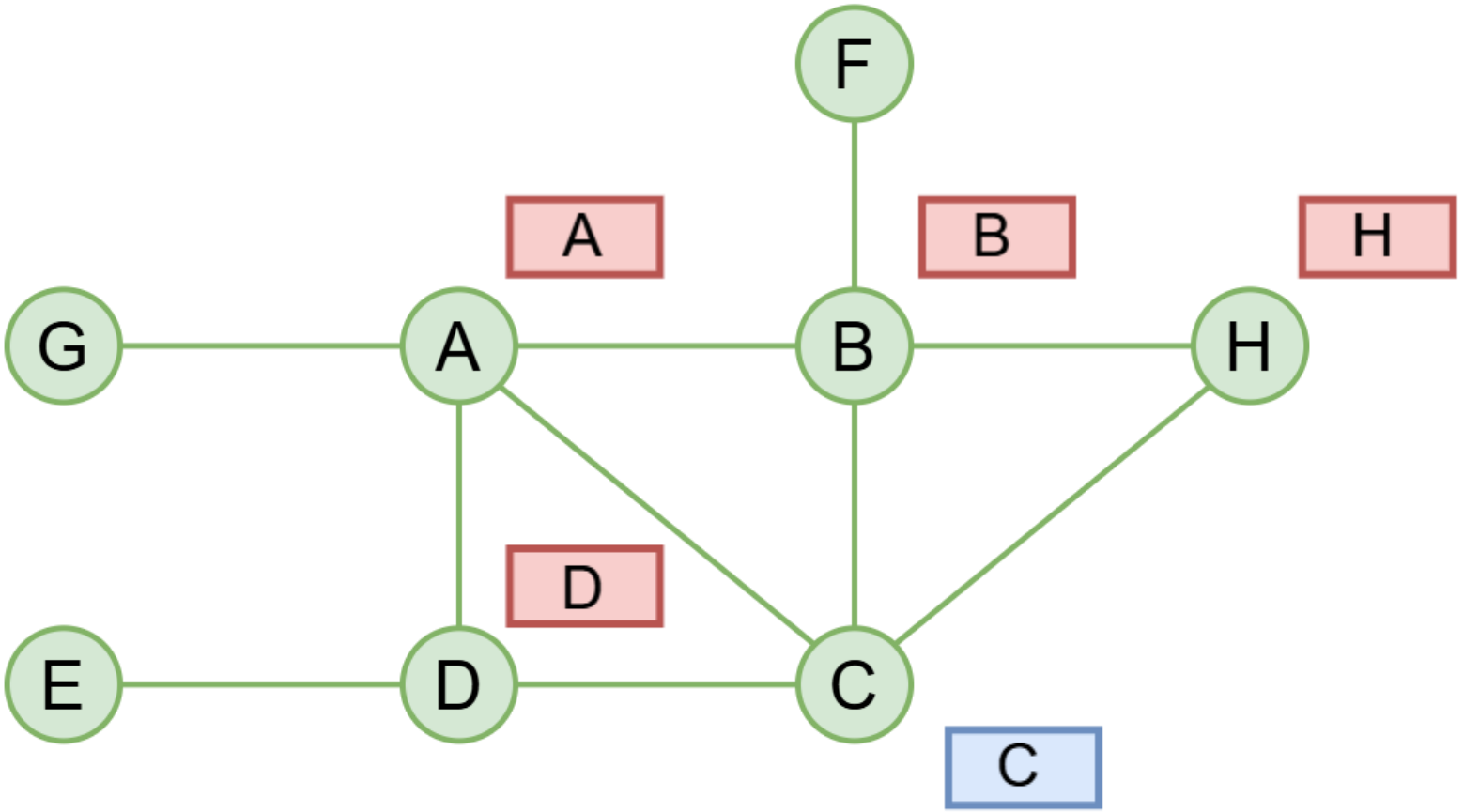
Wu, et al (2022)

How does Message Passing (MP) work?

- To get the node embedding (or representation) of node A, we need the messages (in the form of embeddings) from its neighbors **B, C, and E**.
- Therefore, we first update the representations of **B, C, and E**.
  - For **B**, messages are computed from its neighboring nodes **A, C, and F**.
  - For **C**, messages are computed from its neighboring nodes **A, B, and D**.
  - For **E**, messages are computed from its neighboring nodes **A and F**.
- The messages (MSG) for each node are **aggregated** (AGG).
- The aggregated information is then used to **update** (UPT) each node.

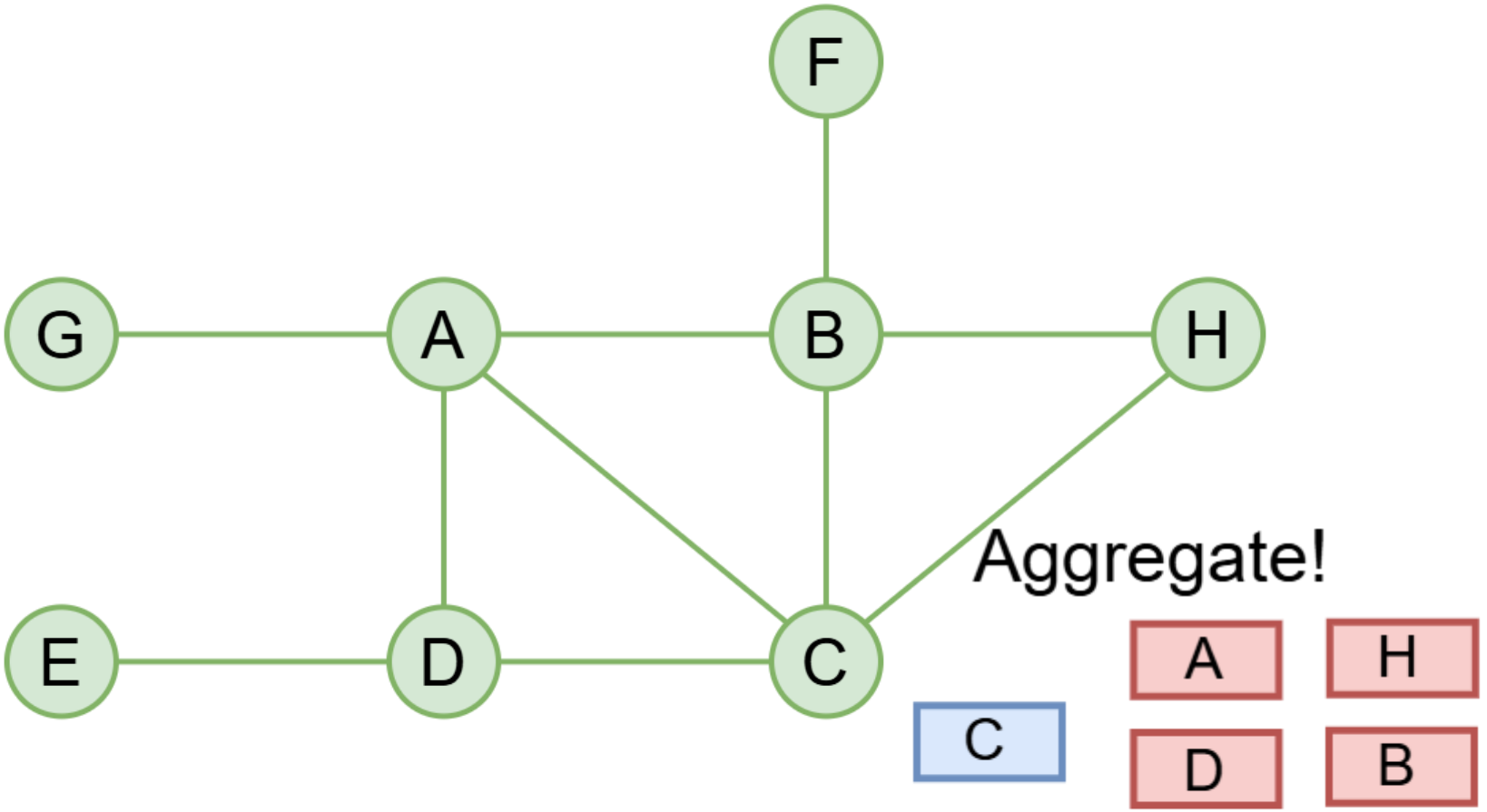


Message Passing Illustration (1-hop)



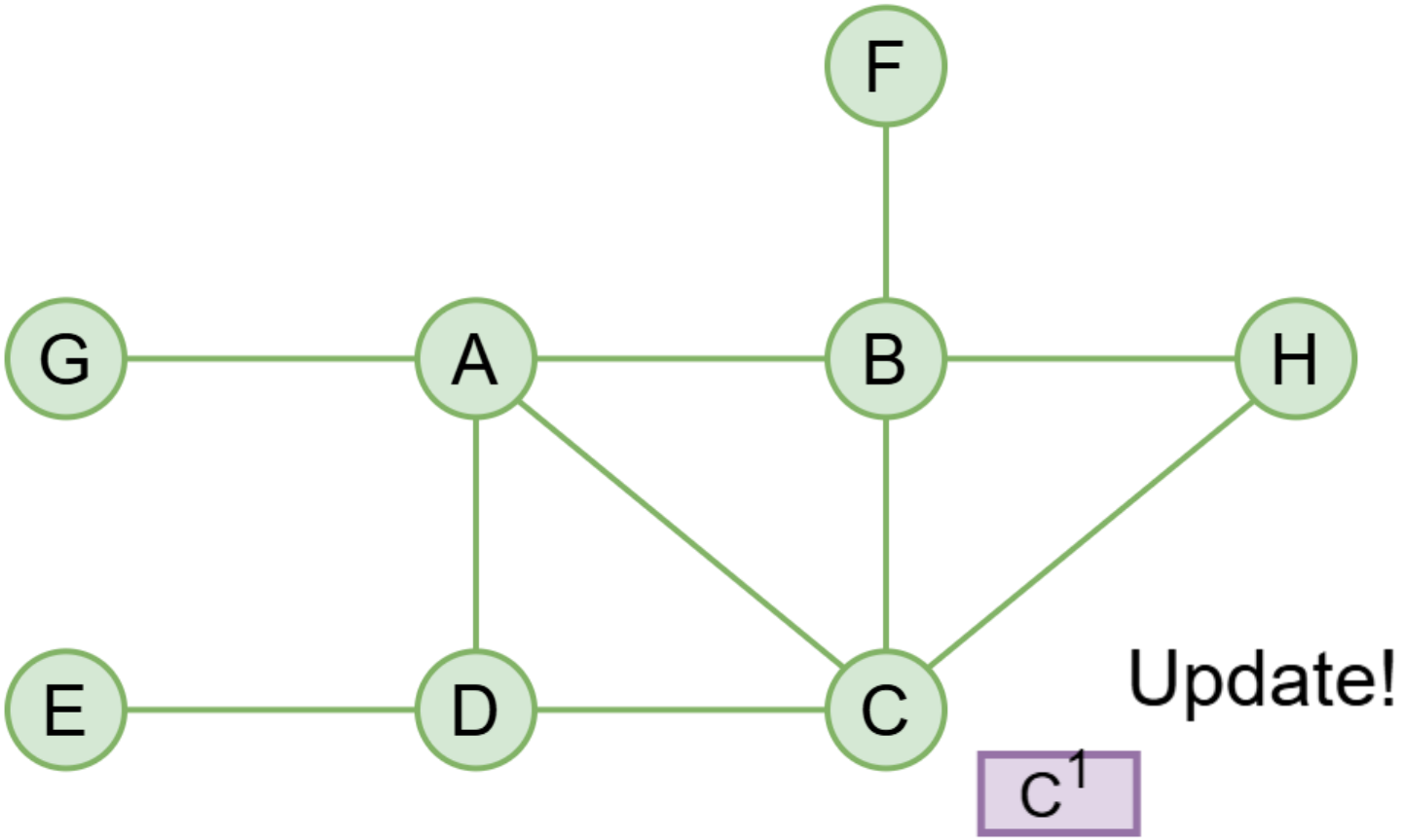
Wu, et al (2022)

Message Passing Illustration (1-hop)



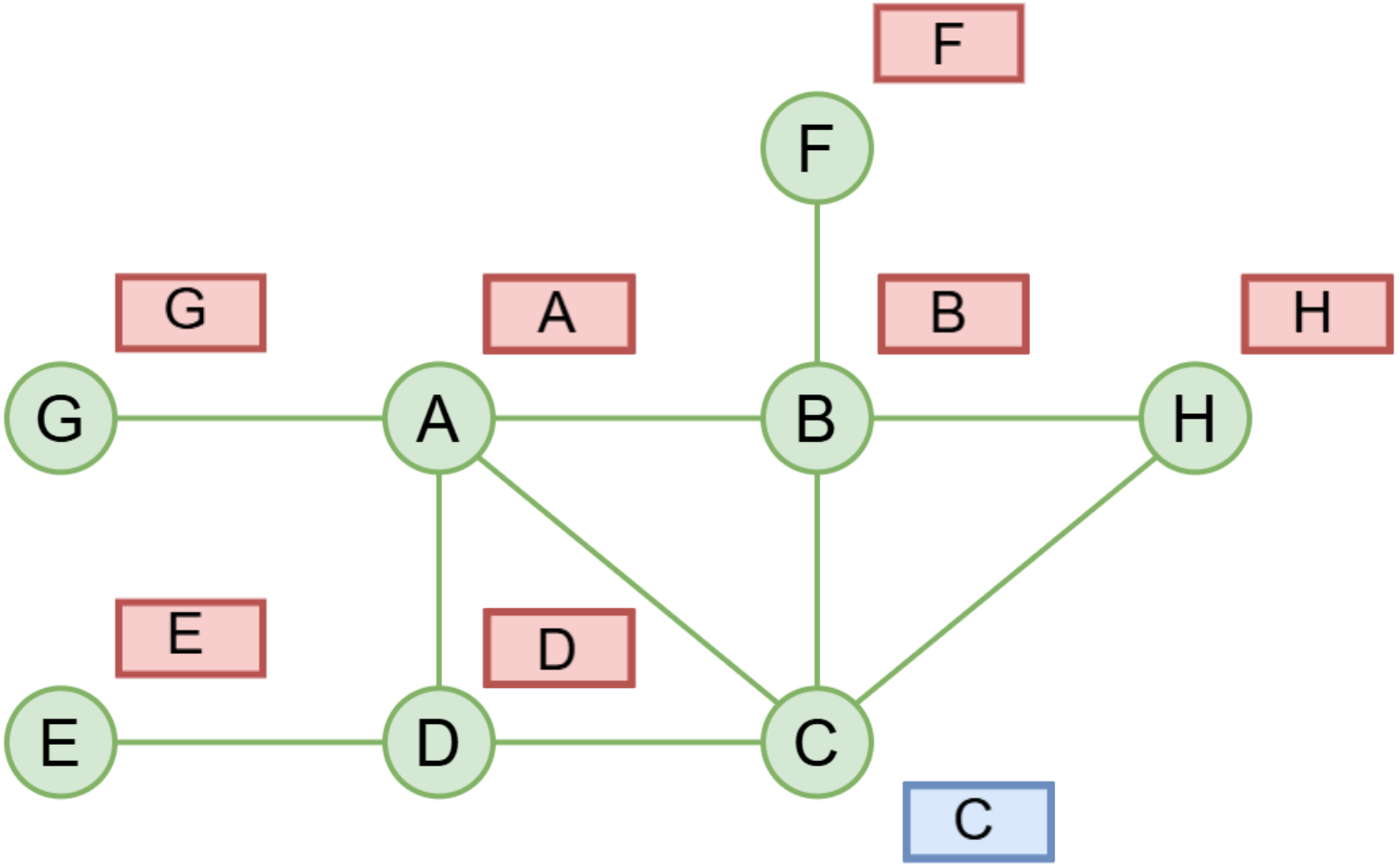
Wu, et al (2022)

Message Passing Illustration (1-hop)



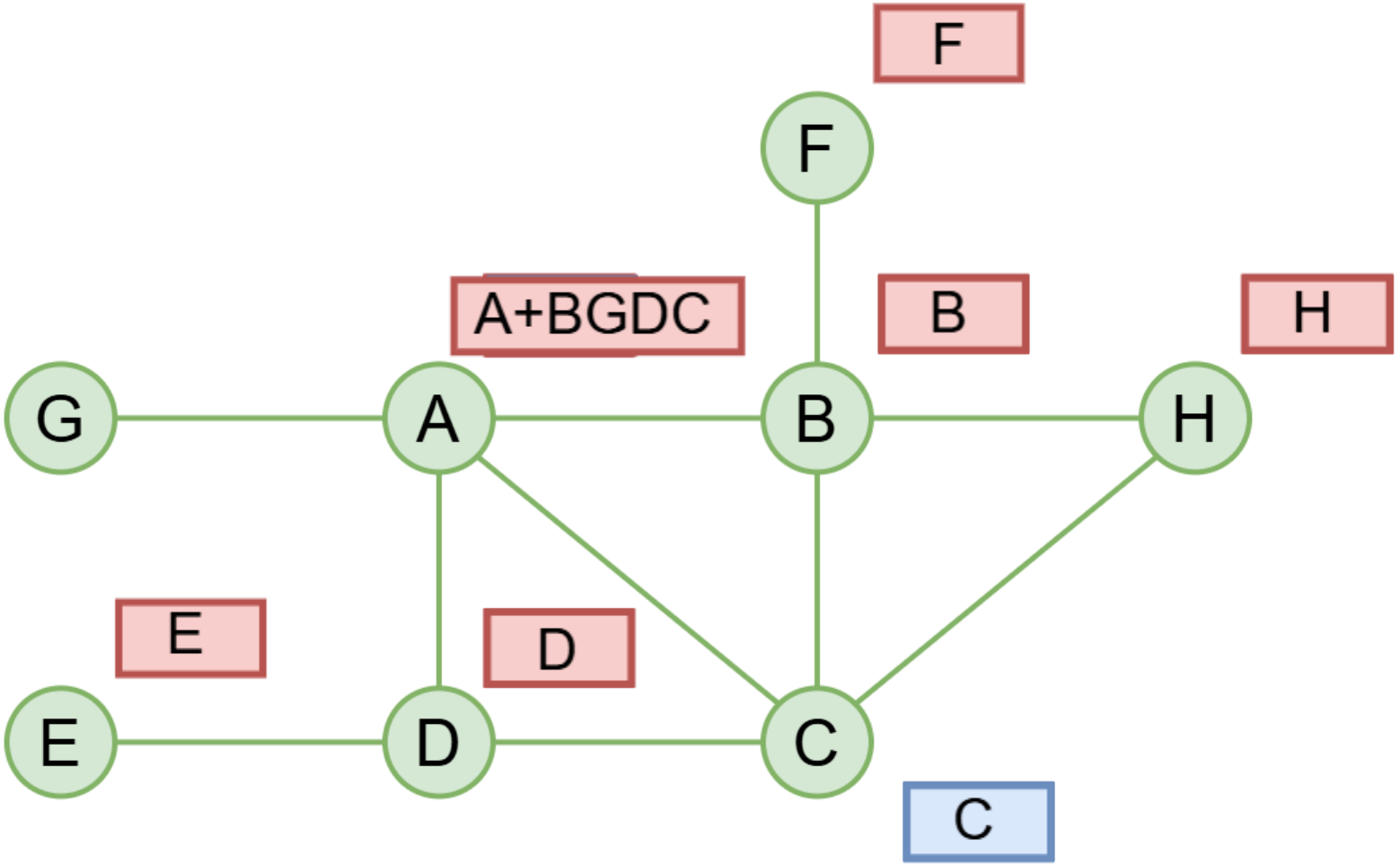
Wu, et al (2022)

Message Passing Illustration (2-hop)



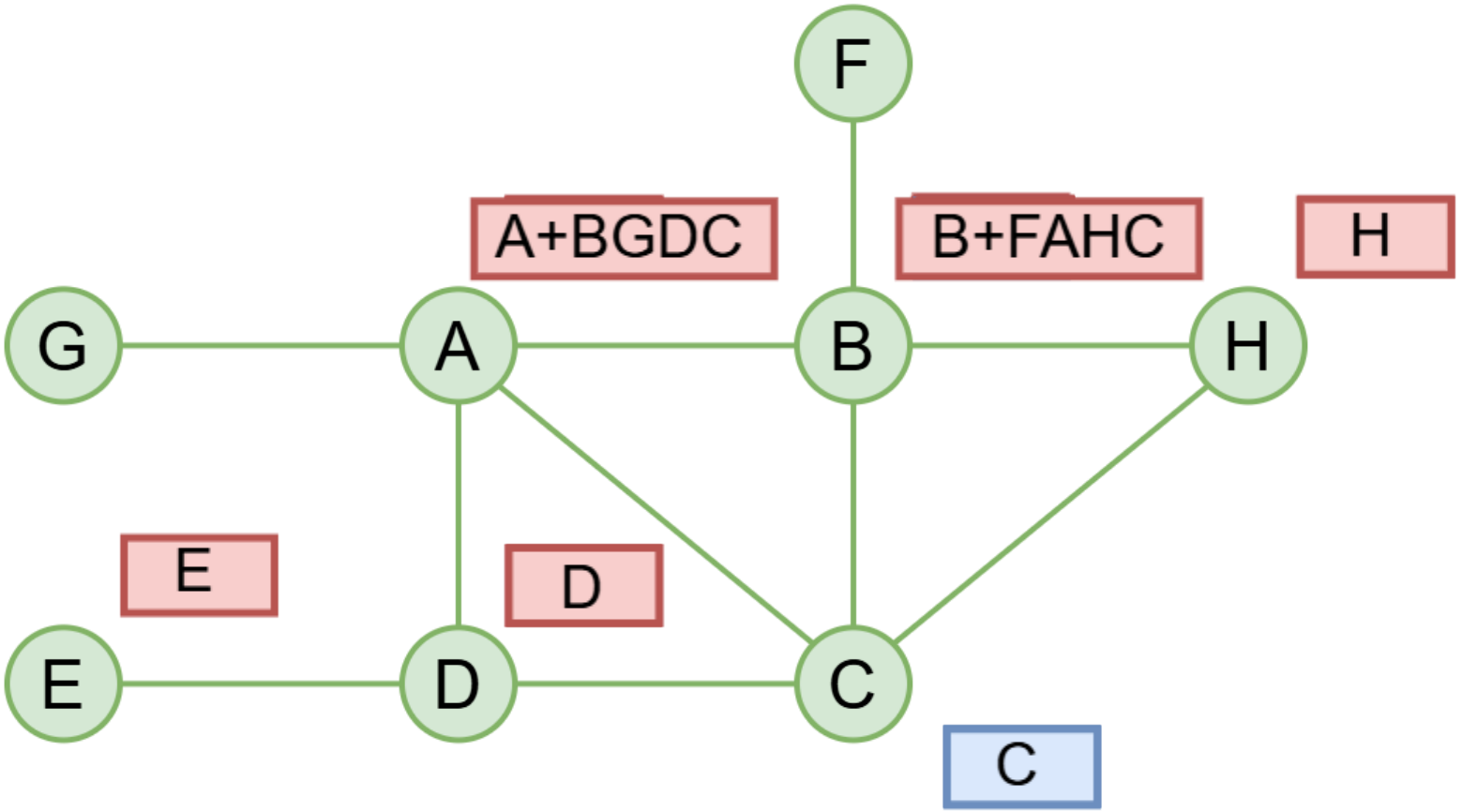
Wu, et al (2022)

Message Passing Illustration (2-hop)



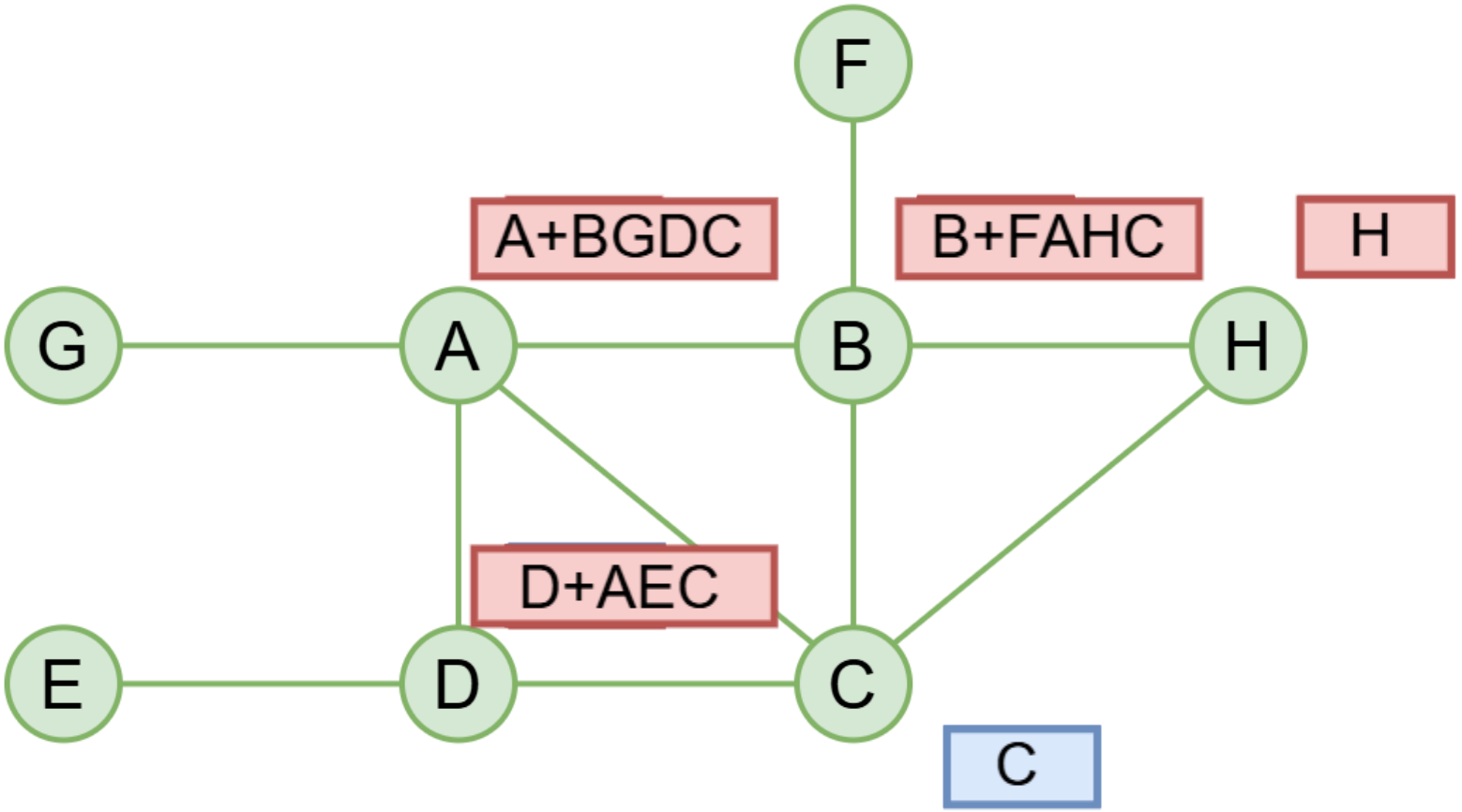
Wu, et al (2022)

Message Passing Illustration (2-hop)



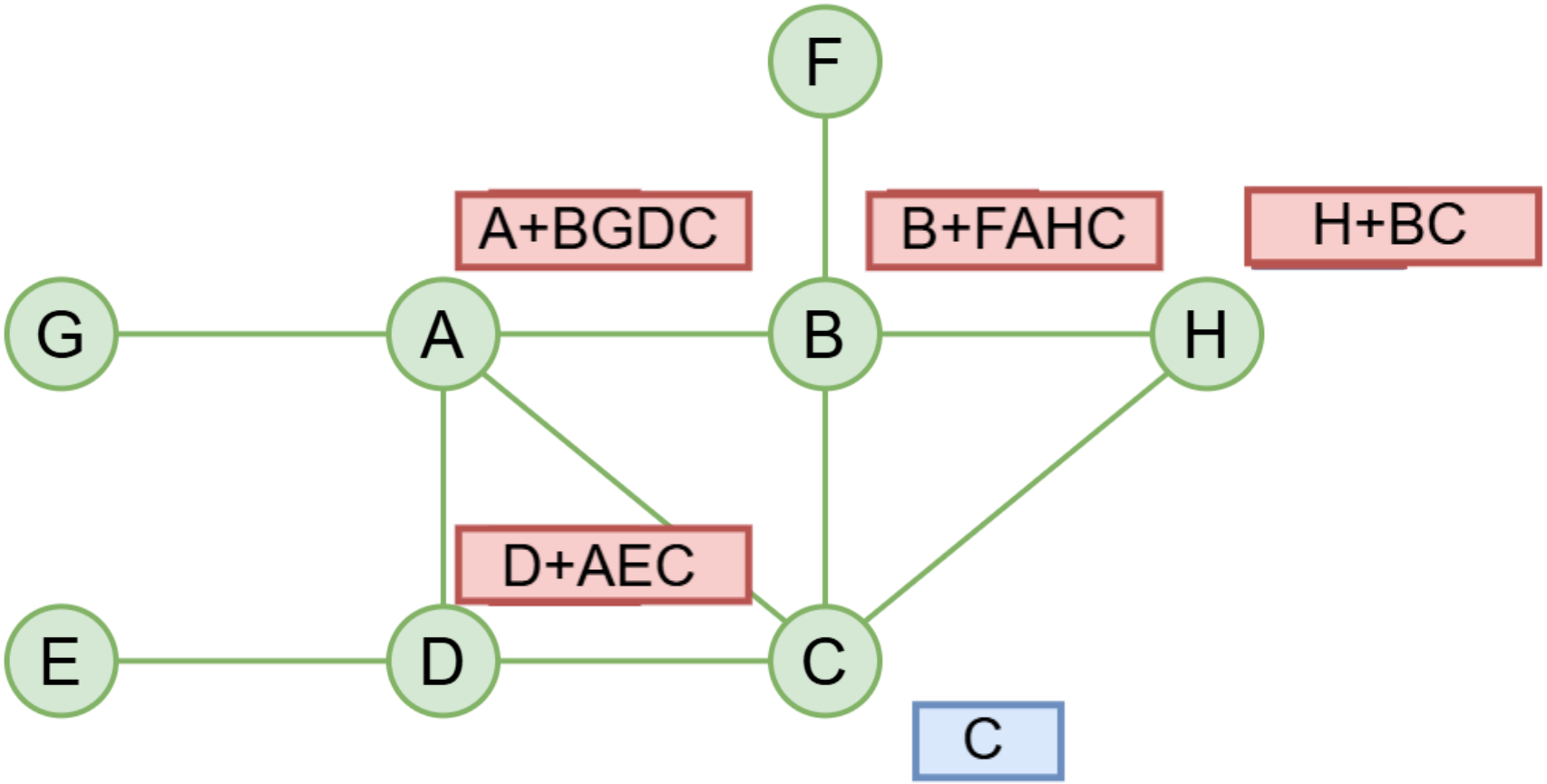
Wu, et al (2022)

Message Passing Illustration (2-hop)



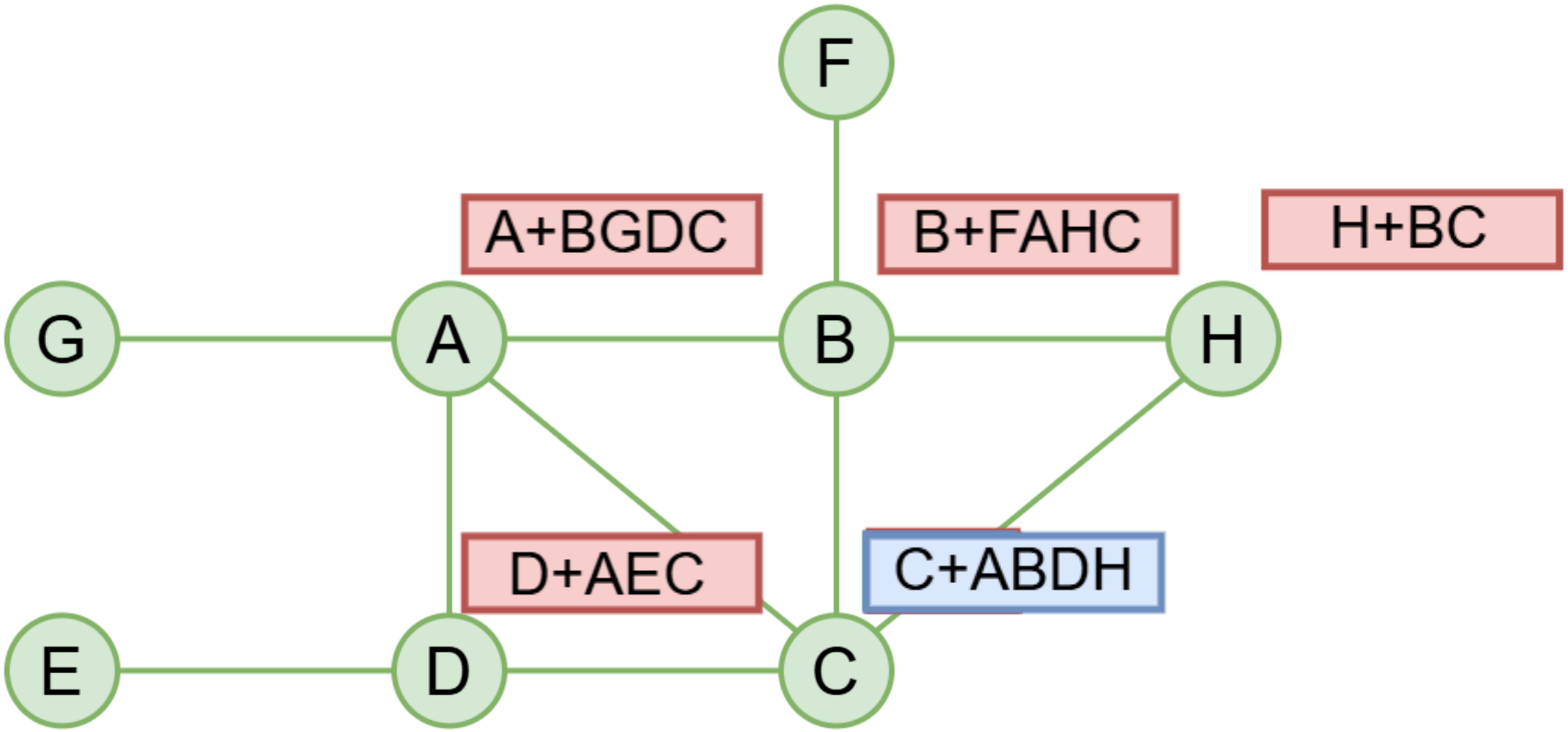
Wu, et al (2022)

Message Passing Illustration (2-hop)



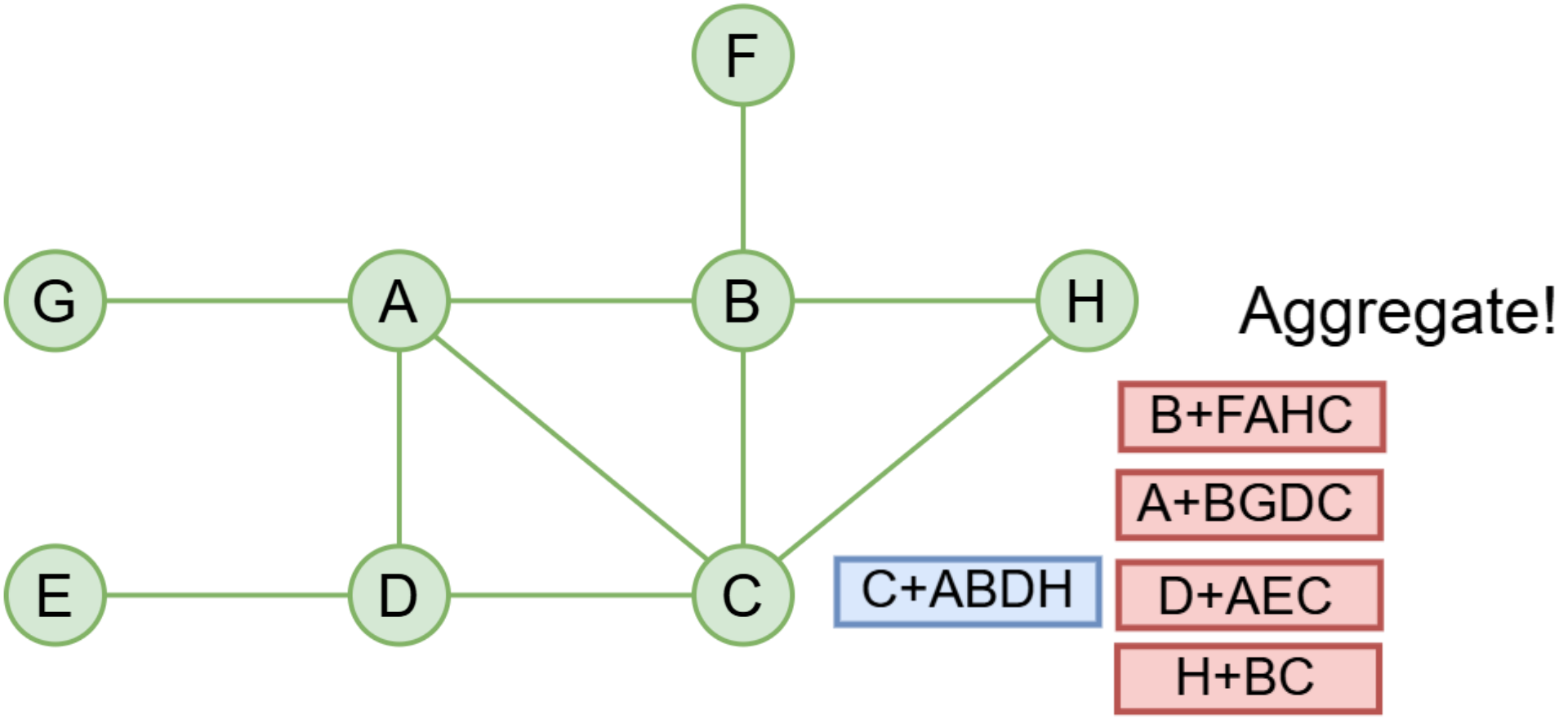
Wu, et al (2022)

Message Passing Illustration (2-hop)

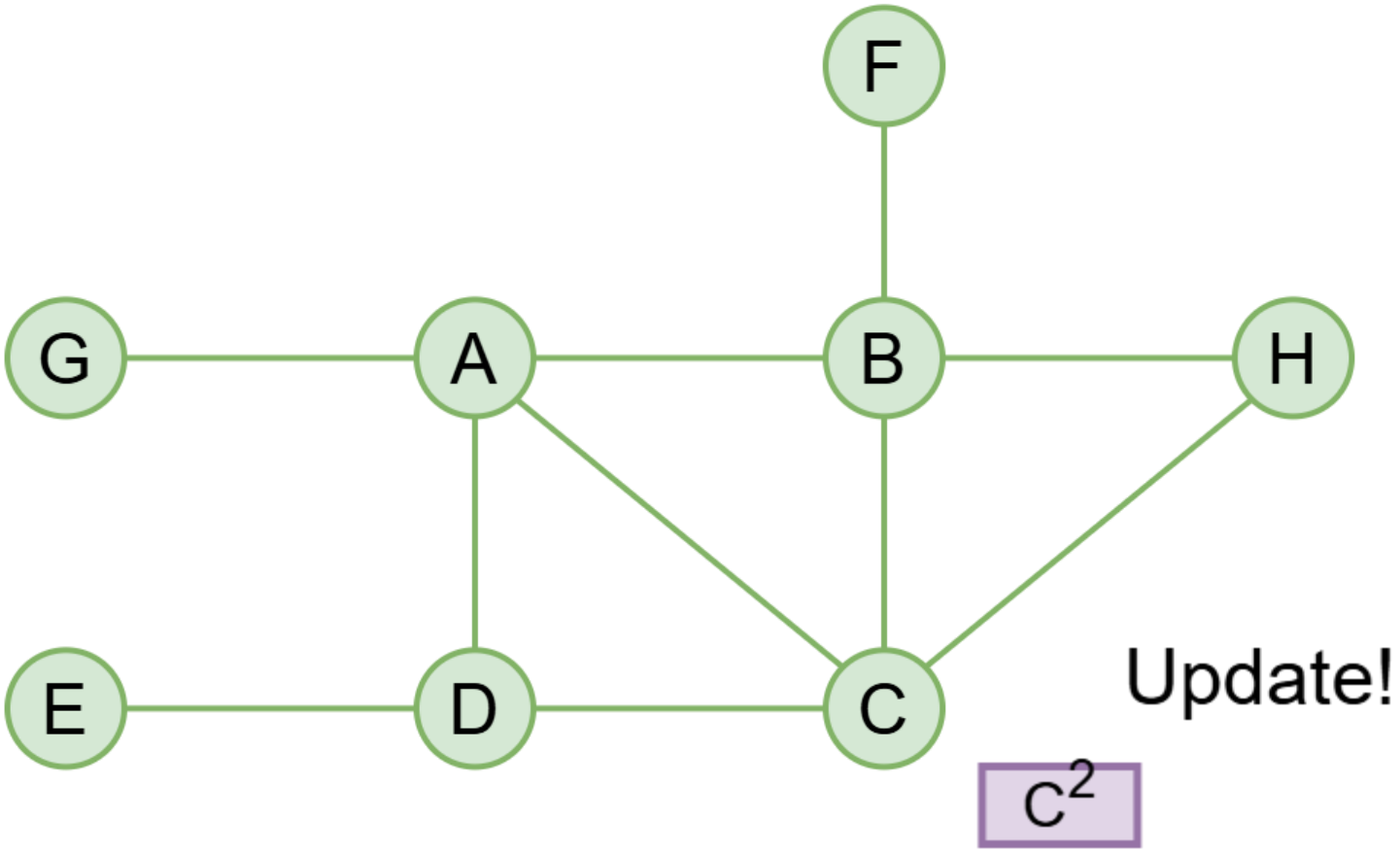


Wu, et al (2022)

Message Passing Illustration (2-hop)



Message Passing Illustration (2-hop)



DL Architecture

GNN architectures: GCN, GraphSAGE, GAT, GIN

GNN Variants

- There are various kinds of GNNs and GNN layers.
  - Each of which has a unique structure and way of aggregating information from neighboring nodes.
- These different variants of GNNs also have their own advantages and limitations and are well-suited for **specific types of graph data** and **tasks**.
- When selecting the appropriate GNN architecture for a particular problem, it is crucial to understand:
  - the characteristics of the graph data and
  - the desired outcome (tasks).

GNN Variants (cont.)

- **Graph convolutional networks (GCN)**
  - It uses every neighbor in aggregate function.
  - Can be used for node classification and node regression (predict a continuous value for each node).
- **Graph SAmple and aggreGatE (GraphSAGE)**
  - It uses only a sample of neighbors in aggregate function.
  - Can be used for node classification, multi-label node classification (every node can have more than 1 class).
- **Graph Attention Network (GAT)**
  - It uses self-attention weights.
  - Can be used for node classification.
- **Graph Isomorphism Network (GIN)**
  - It uses approximation.
  - Can be used for graph classification and node classification.

Han, et al (2024); Wu, et al (2022)

GNN Variants (cont.)

- **Graph Autoencoder** (**GAE**) is one of the GNN variants and a popular link prediction technique.
- The goal of link prediction is to predict a probability (between 0 and 1) for each element of the adjacency matrix, $\hat{A}$.
- The GAE is composed of two modules:
  - The **encoder** is a two-layer GCN that computes node embeddings.
  - The **decoder** approximates the adjacency matrix, $\hat{A}$,
    - using matrix factorization and a sigmoid function, σ , to output probabilities.
    - The idea: similar nodes (thus similar embeddings) should be connected.